Lab Sheet-05

Object Oriented Programming Using Java

LastDate:21-10-2026

MCA III Semester

Instructions for Every Program

- One question = one Java program = one .java file.
- Use exactly ONE public class. The public class contains main() and its name matches the filename.
- Create all required domain/application classes as non-public classes in the same file.
- main() should only create objects, create arrays, call methods, and coordinate the program. Do not put business logic in main().
- Use the exact class roles, variable types, methods, constructors, and access specifiers stated in each question.
- Use this where specified, especially when constructor/setter parameters have the same names as instance variables.
- Use super() where specified to initialize the parent/super class part of an object.
- Do not use interfaces in this lab. Multiple and hybrid inheritance are concept-only where stated.
- Add short natural comments near newly introduced concepts in your code.

Terminology Used in the Questions

Term	Meaning	Example
Instance variable	Belongs to an object; each object can have its own value.	private String name;
Class variable	Shared by all objects; declared static.	static int count;
Local variable	Declared inside a method/constructor/block.	double total = 0;
Method	Defines an operation/behavior.	void display() { }
Constructor	Initializes an object; same name as class and no return type.	Student(...) { }
this	Refers to the current object.	this.name = name;
super class	Parent/base class from which another class inherits.	class Employee
child class	Subclass that inherits from a super class.	class Pharmacist extends Employee
default access	No access keyword; package-	int rollNo;

	private.	
protected	Accessible in the same package and through inheritance in subclasses.	protected int id;

20 Unsolved Questions

Q1. Student Record — Class and Object

Create a Java program for a Student Record System. The program must create a Student class, create a student object, store the student's basic information, and display that information through a class method. Keep the Main class separate from the Student class conceptually while keeping both classes in the same Java file.

- Public Main class: StudentMain.
- Create non-public class: Student.
- Instance variables in Student: String name, int rollNo, double marks.
- Create a no-argument constructor Student() and initialize suitable values.
- Create method displayDetails() to print all Student data.
- In main(), create one Student object and call displayDetails().
- Do not put display logic in main().

Q2. Medicine — Parameterized Constructor and this

Create a Java program for a Medicine Information System. The program must create Medicine objects using a parameterized constructor, initialize each medicine's information, and display the information through a method. Use the this keyword while initializing the object data.

- Public Main class: MedicineMain.
- Create non-public class: Medicine.
- Instance variables: String medicineName, String batchNo, double price.
- Create constructor Medicine(String medicineName, String batchNo, double price).
- Use this.medicineName, this.batchNo and this.price inside the constructor.
- Create displayMedicine().
- In main(), create two Medicine objects using the parameterized constructor and call displayMedicine().

Q3. Pharmacy — Instance and Class Variables

Create a Java program for a Pharmacy Registration System. The program must create multiple Pharmacy objects, store information specific to each pharmacy, maintain one shared count of all Pharmacy objects created, and display both individual pharmacy information and the shared count.

- Public Main class: PharmacyMain.
- Create non-public class: Pharmacy.

- Instance variables: String pharmacyName and String location.
- Class variable: static int pharmacyCount.
-
- Create constructor Pharmacy(String pharmacyName, String location).
- Increment pharmacyCount inside the constructor.
- Create displayPharmacy() and displayPharmacyCount().
- In main(), create three Pharmacy objects and call the methods.
- Do not create pharmacyCount inside main(); it must belong to the class.

Q4. Patient Records — Object Array, Loop and Local Variable

Create a Java program for a Patient Record System. The program must create five Patient objects, store them in an object array, and display all patient records using a loop through a separate manager/processing class.

- Public Main class: PatientMain.
- Create non-public class: Patient.
- Instance variables: int patientId, String patientName, int age.
- Create constructor Patient(int patientId, String patientName, int age).
- Create displayPatient().
- Create non-public class PatientManager.
- Create method displayAllPatients(Patient[] patients) in PatientManager.
- Inside displayAllPatients(), use a local loop variable int i and call displayPatient() for every object.
- In main(), create a Patient[] array of five objects and pass it to PatientManager.
- Do not put the processing loop in main().

Q5. Medicine Eligibility — Method and Local Variable

Create a Java program that checks whether a patient is eligible to use a particular medicine based on an age limit. The eligibility decision must be performed by a method, which returns the result, and the method must use a local variable to store that result.

- Public Main class: EligibilityMain.
- Create non-public class: Medicine.
- Instance variables: String medicineName and int ageLimit.
- Create constructor Medicine(String medicineName, int ageLimit).
- Create method String checkEligibility(int patientAge).
- Inside checkEligibility(), create local variable String result.
- Use if-else to assign Eligible/Not Eligible to result and return it.
- In main(), create one Medicine object and test at least two patient ages.
- The age comparison must be inside checkEligibility().

Q6. Encapsulated Medicine Product

Create a Java program for an encapsulated Medicine Product. The medicine's internal data must be protected from direct access by making the data private. Provide public getters and setters so that the data can be read and modified in a controlled way, and display the product information.

- Public Main class: MedicineProductMain.
- Create non-public class: MedicineProduct.
-
- Private instance variables: String medicineName, String batchNo, double price.

- Create parameterized constructor MedicineProduct(String medicineName, String batchNo, double price).
- Use this for all three instance variables.
- Create public getters for all three variables.
- Create public setters for all three variables.
- Create displayProduct().
- In main(), create an object, update at least one field through a setter, and read at least one field through a getter.
- Do not directly access the private variables from main().

Q7. Encapsulated Patient — Validation

Create a Java program for an encapsulated Patient Record. The patient's internal data must be private, and setters must control whether new age and weight values are accepted. Demonstrate valid and invalid updates and display the accepted patient information.

- Public Main class: PatientMain.
- Create non-public class: Patient.
- Private instance variables: String patientName, int age, double weight.
- Create parameterized constructor Patient(String patientName, int age, double weight).
- Create public getters and setters for all three fields.
- setAge(int age) accepts a value only when age >= 0.
- setWeight(double weight) accepts a value only when weight > 0.
- Use this.age and this.weight inside the setters.
- Create displayPatient().
- In main(), demonstrate both valid and invalid setter values.
- Do not directly access private fields from main().

Q8. Bank Account — Encapsulation, this and Local Variable

Create a Java program for a Bank Account. The account must keep its data encapsulated, initialize itself through a parameterized constructor, support deposit and withdrawal operations, prevent an invalid withdrawal when the balance is insufficient, and display the account balance. Use this and a local variable as specified below.

- Public Main class: BankAccountMain.
- Create non-public class: BankAccount.

- Private instance variables: String accountNo, String accountHolderName, double balance.
- Create constructor BankAccount(String accountNo, String accountHolderName, double balance) and use this.
- Create deposit(double amount).
- Create withdraw(double amount).
- Inside withdraw(), create local variable boolean success.
-
- Withdrawal must fail when amount is greater than balance.

- Create getBalance() and displayAccount().
- In main(), create one account and call deposit(), withdraw(), and displayAccount().
- All account rules/calculations must be inside BankAccount methods.

Q9. Public Access Specifier

Create a Java program that demonstrates the public access specifier. Create a Hospital class whose selected data and method are public, then create a Hospital object from Main and directly access the public members.

- Public Main class: PublicAccessMain.
- Create non-public class: Hospital.
- Create public instance variables: String hospitalName and String hospitalCode.
- Create public no-argument constructor Hospital().
- Create public method displayHospital().
- In main(), create a Hospital object and directly access hospitalName and hospitalCode.
- Call displayHospital().
- Add a short comment near the public members about their access.

Q10. Private Access Specifier

Create a Java program that demonstrates the private access specifier. Create an Employee class whose data is private and show that Main cannot directly access those fields. Provide public getter methods so Main can read the data in a controlled way.

- Public Main class: PrivateAccessMain.
- Create non-public class: Employee.
- Private instance variables: int employeeId, String employeeName, double salary.
- Create parameterized constructor Employee(int employeeId, String employeeName, double salary).
- Create public getters for all three private variables.
- Create public displayEmployee().
- In main(), access the information only through getters/displayEmployee().
- Add a commented-out direct access example such as employee.salary to show that it is not allowed.

- Do not change the variables to public.

Q11. Default Access Specifier

Create a Java program that demonstrates default/package-private access. Create a Pharmacy class whose selected members have no access-specifier keyword, then access those members from Main because both classes are in the same package.

- Public Main class: DefaultAccessMain.
- Create non-public class: Pharmacy.
- Create default-access instance variables String medicineName and double price. Do not write an access keyword.
- Create default-access constructor Pharmacy().
- Create default-access method displayMedicine().
- In main(), create Pharmacy and access the default members directly because both classes are in the same package.
- Add comments identifying the declarations as default/package-private.
- Do not use public/private/protected for the members requested as default.

Q12. Protected Access Specifier

Create a Java program that demonstrates protected access through inheritance. Create a HospitalStaff super class with protected data and a protected method, then create a Pharmacist child class that inherits from HospitalStaff and accesses the protected members.

- Public Main class: ProtectedAccessMain.
- Create super class: HospitalStaff.
- HospitalStaff protected instance variables: int staffId and String staffName.
- Create protected constructor HospitalStaff(int staffId, String staffName).
- Create protected method displayStaff().
- Create child class: Pharmacist extends HospitalStaff.
- Pharmacist private instance variable: String licenseNo.
- Create Pharmacist(int staffId, String staffName, String licenseNo).
- Inside Pharmacist constructor, call super(staffId, staffName).
- Use this.licenseNo for the child variable.
- Inside a Pharmacist method, access inherited protected staffId and staffName.
- In main(), create Pharmacist and call its method.

Q13. Single Inheritance — Employee → Pharmacist

Create a Java program demonstrating single inheritance in which Employee is the super class and Pharmacist is the child class. The Employee class must contain common employee information, while Pharmacist adds pharmacy-specific information. Initialize both levels through constructors and display the complete Pharmacist record.

- Public Main class: SingleInheritanceMain.

- Super class: Employee.
- Employee protected instance variables: int employeeId, String employeeName, double salary.
- Create Employee(int employeeId, String employeeName, double salary).
- Use this in the Employee constructor.
- Create Employee method displayEmployee().
- Child class: Pharmacist extends Employee.
- Pharmacist private instance variable: String licenseNo.
- Create Pharmacist(int employeeId, String employeeName, double salary, String licenseNo).
- Call super(employeeId, employeeName, salary) from the Pharmacist constructor.
- Use this.licenseNo for the child variable.
- Create displayPharmacist() showing inherited and child data.
- In main(), create a Pharmacist object and call displayPharmacist().
- Clearly identify Employee as super class and Pharmacist as child class.

Q14. Hierarchical Inheritance — Person → Doctor and Nurse

Create a Java program demonstrating hierarchical inheritance. Person must be the super class, and Doctor and Nurse must be two separate child classes. Both child classes must inherit common personal information from Person and add their own role-specific information.

- Public Main class: HierarchicalMain.
- Super class: Person.
- Person protected instance variables: String name and int age.
- Create Person(String name, int age) and use this.name and this.age.
- Create displayPerson().
- Child class 1: Doctor extends Person.
- Doctor private instance variable: String specialization.
- Create Doctor(String name, int age, String specialization) and call super(name, age).
- Child class 2: Nurse extends Person.
- Nurse private instance variable: String ward.
- Create Nurse(String name, int age, String ward) and call super(name, age).
- Create displayDoctor() and displayNurse().
- In main(), create one Doctor and one Nurse.
- Clearly identify Person as super class and Doctor/Nurse as child classes.

Q15. Multilevel Inheritance — Person → Employee → Manager

Create a Java program demonstrating multilevel inheritance using the chain Person → Employee → Manager. Person must be the top-level super class, Employee must inherit from Person and become the intermediate super class for Manager, and Manager must be the final child class. Initialize all three levels through constructors.

- Public Main class: MultilevelMain.
- Level 1 super class: Person with protected instance variables String name and int age.
- Person constructor: Person(String name, int age).
- Level 2 class: Employee extends Person. Employee is a child of Person and also a super class for Manager.
- Employee private instance variable: int employeeId.
- Employee constructor: Employee(String name, int age, int employeeId). Call super(name, age).
- Level 3 child class: Manager extends Employee.
- Manager private instance variable: int teamSize.
- Manager constructor: Manager(String name, int age, int employeeId, int teamSize). Call super(name, age, employeeId).
- Use this.employeeId and this.teamSize where appropriate.
- Create displayManager().
- In main(), create one Manager object.
- Clearly identify Person as top-level super class, Employee as intermediate child/super class, and Manager as final child class.

Q16. Protected Parent + Private Child Data

Create a Java program comparing protected inherited data with private child data. Create MedicalStaff as the super class and Doctor and Pharmacist as child classes. The child classes must directly use inherited protected data while keeping their own additional data private.

- Public Main class: MedicalStaffMain.
- Super class: MedicalStaff.
- MedicalStaff protected instance variables: int staffId and String staffName.
- Create MedicalStaff(int staffId, String staffName).
- Child class 1: Doctor extends MedicalStaff.
- Doctor private instance variable: String specialization.
- Create Doctor(int staffId, String staffName, String specialization), using super() and this.
- Create displayDoctor() that accesses inherited protected fields and Doctor's private field.
- Child class 2: Pharmacist extends MedicalStaff.
- Pharmacist private instance variable: String licenseNo.
- Create Pharmacist(int staffId, String staffName, String licenseNo), using super() and this.
- Create displayPharmacist().

- In main(), create one Doctor and one Pharmacist.
- Do not use interfaces.

Q17. Multiple and Hybrid Inheritance — Concept Only

Create a Java program that explains, through comments, multiple inheritance and hybrid inheritance in Java. Explain why a Java class cannot extend two classes and why interfaces are relevant to multiple/hybrid inheritance. Do not implement an interface or write executable multiple-inheritance code.

- Public Main class: InheritanceConceptMain.
- Use comments to explain multiple inheritance of classes.
- Explain why Java does not allow a class to extend two classes.
- Show the invalid idea only inside a comment: class C extends A, B.
- Use comments to explain hybrid inheritance as a combination of inheritance forms.
- State that interfaces are commonly used when Java needs multiple/hybrid inheritance of type/behavior.
- Do not create or implement an interface.
- Do not write executable multiple-inheritance code.

Q18. Shape Hierarchy — Parent Reference Array

Create a Java program demonstrating hierarchical inheritance with Shape as the super class and Circle and Rectangle as child classes. Store Circle and Rectangle objects in a Shape array and process them through a separate manager class using a loop. Each child class must calculate its own area.

- Public Main class: ShapeMain.
- Super class: Shape.
- Shape protected instance variable: String name.
- Create Shape(String name) and use this.name.
- Create displayShapeName().
- Child class: Circle extends Shape with private instance variable double radius.
- Create Circle(String name, double radius), call super(name), and use this.radius.
- Create calculateArea() in Circle.
- Child class: Rectangle extends Shape with private instance variables double length and double width.
- Create Rectangle(String name, double length, double width), call super(name), and use this.length/this.width.
- Create calculateArea() in Rectangle.
- In main(), create a Shape[] containing a Circle and a Rectangle.
- Create non-public ShapeManager with displayAreas(Shape[] shapes).

- Inside displayAreas(), use a loop and identify each child object before calling the appropriate area method.
- Use a local loop variable in displayAreas().
- Do not calculate areas in main().

Q19. Hospital Staff Management — Integrated OOP

Create an integrated Hospital Staff Management System using Person, Employee, Doctor, and Pharmacist classes. The system must combine encapsulation, access specifiers, constructors, this, super(), inheritance, static data, arrays, loops, conditions, getters/setters, and methods while keeping business logic outside main().

- Public Main class: HospitalStaffMain.
- Super class: Person with protected instance variables String name and int age.
- Create Person(String name, int age) using this.
- Child class: Employee extends Person.
- Employee private instance variables: int employeeId and double salary.
- Create Employee(String name, int age, int employeeId, double salary) using super(name, age).
- Child class: Doctor extends Employee with private String specialization.
- Create Doctor(...) using super() and this.
- Child class: Pharmacist extends Employee with private String licenseNo.
- Create Pharmacist(...) using super() and this.
- Create getters/setters for private fields where needed.
- Create static class variable employeeCount in Employee and update it when an Employee object is constructed.
- Create a method in Employee or a separate non-public manager class to process/display staff.
- In main(), create an Employee[] containing Doctor and Pharmacist objects.
- Use a loop to display records.
- Use at least one condition to filter/identify role or department.
- Keep validation, calculations, and filtering outside main().
- Clearly identify Person as top-level super class and Doctor/Pharmacist as descendants through Employee.

Q20. Final Challenge — Pharmacy Employee Management System

Create a complete Pharmacy Employee Management System using all major concepts learned in this lab. Employee must be the super class, Pharmacist and StoreManager must be child classes, and the system must demonstrate encapsulation, access control, constructors, this, super(),

instance/class/local variables, methods, inheritance, arrays, loops, conditions, and controlled data access.

- Public Main class: PharmacyEmployeeMain.
- Super class: Employee.
- Employee private instance variables: int employeeId, String employeeName, double salary.
- Employee class variable: static int employeeCount shared by all Employee objects.
- Create Employee(int employeeId, String employeeName, double salary); use this for instance variables and increment employeeCount.
- Create public getters and setters for Employee private fields.
- Salary setter must reject negative salary values.
- Child class 1: Pharmacist extends Employee.
- Pharmacist private instance variable: String licenseNo.
- Create Pharmacist constructor with all required data; call super(employeeId, employeeName, salary) and use this.licenseNo.
- Child class 2: StoreManager extends Employee.
- StoreManager private instance variable: int storeSection.

- Create StoreManager constructor with all required data; call super(employeeId, employeeName, salary) and use this.storeSection.
- Use at least one protected member where inheritance genuinely benefits from it.
- Use at least one default-access member or method; do not write an access keyword for it.
- Create at least one local variable inside a processing method, such as a total, counter, or status.
- Create suitable display and role-specific methods.

- In main(), create an Employee[] containing Pharmacist and StoreManager objects.
- Create a separate non-public manager/utility class to process the Employee[] array.
- Use a loop to process all employees.
- Use at least one condition to identify role or validate/filter data.
- Use getters/setters for controlled access to encapsulated data.
- Use super() for parent initialization and this for current-object initialization.
- main() must only create objects/arrays and call methods; all business logic must be outside main().
- Do not use interfaces.
- Add short comments beside newly introduced concepts.
- Final program must demonstrate class, object, instance variable, class/static variable, local variable, method, constructor, this, encapsulation, public/private/default/protected access, inheritance, and super().

Submission Checklist

☐ Exactly one public class.

☐ Filename matches the public Main class.

☐ All required classes are in the same file.

☐ Super class and child class roles match the question.

☐ Instance/class/static/local variables match the question.

☐ Required methods and constructors are present.

☐ this and super() are used where specified.

☐ Access specifiers match the question.

☐ main() contains no business logic.

☐ No interface is used.

☐ Program compiles and runs successfully.